COMPLETE VISUAL TUTORIAL

XGBoost Hyperparameters

From Zero to Pro

How XGBoost works from scratch · what every dial does · the mathematics made simple · tuning, speed and full working code



Inside	What you get
Part 1 · Basics	decision trees, boosting by hand, over/underfitting, what a hyperparameter is
Part 2 · The Dials	all 9 key parameters in 4 families + the grid-search cheat sheet
Part 3 · Big Three	learning_rate/n_estimators, max_depth, min_child_weight (+ sampling)
Part 4 · Mathematics	objective, Taylor expansion, leaf weight, Gain, worked numbers, XGBoost in 60 lines
Part 5 · Pro	hist, GPU, early stopping, NaNs, imbalance, staged tuning, Optuna, 5 community tips
Part 6–7 · Code & Cheat Sheets	tested templates, troubleshooting table, golden rules, exercises

All charts marked “real” were produced by actually running XGBoost; all code listings were executed before inclusion.

Contents

How to read this tutorial 4

Prerequisites 4

PART 1 · The Basics (Level 1) 5

1.1 The problem: learning from examples 5

1.2 The building block: a decision tree 5

1.3 One tree is not enough 6

1.4 Boosting: learn from your mistakes 6

1.5 So what is XGBoost? 7

1.6 What is a hyperparameter? 8

1.7 Your first XGBoost model (5 lines) 9

PART 2 · The Dials (Level 2) 10

2.1 Family 1 – Core learning parameters 10

2.2 Family 2 – Tree complexity (overfitting control) 11

2.3 Family 3 – Randomness and sampling 12

2.4 Family 4 – Regularisation (a penalty for complexity) 13

2.5 The Grid Search Cheat Sheet 13

PART 3 · The “Big Three” Deep Dive (Level 2+) 15

3.1 learning_rate (eta) and n_estimators 15

3.2 max_depth 16

3.3 min_child_weight 17

3.4 The “Second Tier”: subsample and colsample_bytree 19

3.5 How the big three interact 19

PART 4 · How It Works Mathematically (Level 3) 20

4.1 Notation cheat-sheet 20

4.2 The model is a sum of trees 20

4.3 What are we minimising? The objective 21

4.4 Training one tree at a time (additive training) 21

4.5 The Taylor trick — why XGBoost is “extreme” 21

4.6 Optimal leaf weight 22

4.7 Choosing splits: the Gain formula 22

4.8 Squared error vs logistic loss: what g and h look like 23

4.9 Where each hyperparameter appears in the maths 24

4.10 Finding the best split: three algorithms 24

4.11 Build XGBoost from scratch (60 lines, verified) . . . 24

PART 5 · Pro Level: Speed & Efficiency . . . 26

5.1 Speed and compute efficiency . . . 26
5.2 Data and pipeline efficiency . . . 28
5.3 Tuning workflow efficiency . . . 30
5.4 Community tips — explained like you are five . . . 31

PART 6 · Complete Code (Level 4) . . . 34

6.1 The Kaggle-style GridSearchCV template . . . 34
6.2 The pro workflow: staged tuning + early stopping + DMatrix . . . 36
6.3 Looking inside the final model . . . 36

PART 7 · Cheat Sheets & Practice . . . 37

7.1 Troubleshooting table . . . 37
7.2 Recommended starting recipe . . . 37
7.3 The ten golden rules . . . 37
7.4 Glossary . . . 38
7.5 Practice exercises (do these to really learn) . . . 38

How to read this tutorial

This tutorial was rebuilt from a YouTube-style script. The script's content is kept in full — every parameter, tip, grid value and code listing — but reorganised into a logical path from absolute basics to professional practice, and enriched with diagrams, real experiments and the underlying mathematics.

Level	You will learn	Maths needed
1 Basic	what trees, boosting and hyperparameters are	none
2 Intermediate	every important parameter + the big three	none
3 Advanced	why it works: objective, gain, leaf weights	algebra, derivatives (explained)
4 Pro	speed, efficiency, tuning strategy, full pipelines	none

Coloured boxes appear throughout:

> TIP **TIP**
> A practical shortcut or best practice.

> ANALOGY **ANALOGY**
> A real-life picture of an abstract idea.

> MATH **MATH**
> The formula behind the idea, with plain-English reading.

> WATCH OUT **WATCH OUT**
> A subtlety, trade-off or version difference.

> COMMON MISTAKE **COMMON MISTAKE**
> Something that quietly ruins results.

Prerequisites

Basic Python and a feeling for what “training a model” means. To run the code: `pip install xgboost scikit-learn matplotlib optuna`. Code was tested with XGBoost 3.x and scikit-learn 1.8.

PART 1 · The Basics (Level 1)

Goal of this part: understand **what XGBoost actually is**, with zero maths. By the end you will know what a decision tree is, how boosting turns many weak trees into one strong model, and why we even need “hyperparameters”.

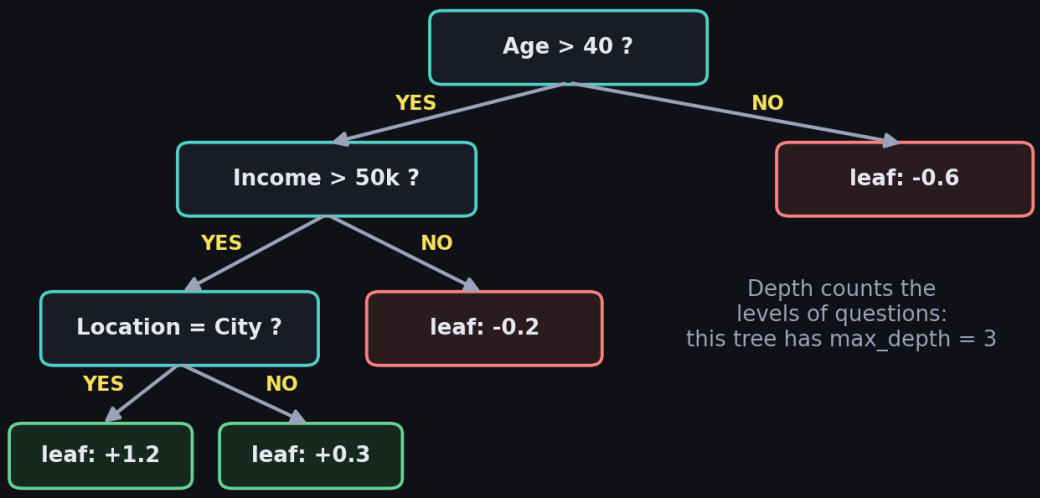
1.1 The problem: learning from examples

Imagine a bank wants to decide whether to approve a loan. For every past customer it has a row of facts (age, income, city, ...) called **features** (written X) and what happened (repaid or defaulted) called the **target** (written y). A machine-learning model studies these examples and learns a rule that predicts y for a new customer it has never seen.

The whole difficulty is **generalisation**: the model must work on unseen data, not just memorise the examples it was shown. Almost everything in this tutorial (every hyperparameter!) is a tool for getting that balance right.

1.2 The building block: a decision tree

A decision tree is a flow-chart of yes/no questions. You start at the top (the **root**), answer a question about one feature, follow the branch, and repeat until you hit a **leaf**. The leaf contains a number — the tree's answer (in XGBoost this number is called the leaf **weight** or score).



Each leaf holds a number (a score / weight). A prediction = the leaf your data row falls into.

Figure 1 – A decision tree of depth 3. A customer aged 45, income 70k, living in a city lands in the leaf “+1.2”.

- **Root / node:** a question such as “Age > 40?”.
- **Branch:** the YES / NO answer that sends a row left or right.
- **Leaf:** the end of a path; holds the predicted value (+1.2, −0.6, ...).
- **Depth:** how many levels of questions deep the tree goes. The tree above has `max_depth = 3`.
- **Interaction:** a path like “Age > 40 AND Income > 50k AND Location = City” lets a tree capture combinations of features. Deeper trees can learn more specific interactions.

1.3 One tree is not enough

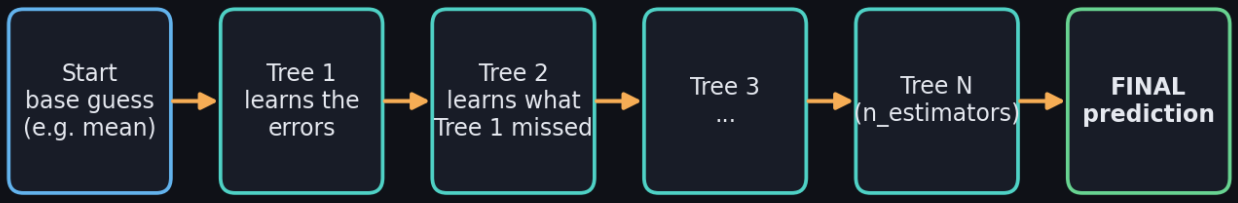
A single small tree is a **weak learner**: easy to understand but not very accurate. A single huge tree memorises the training data (it becomes a lookup table) and fails on new data. The solution used by all modern tree methods: **combine many trees**. There are two famous ways to do it:

	Bagging (Random Forest)	Boosting (XGBoost)
How trees are built	Independently, in parallel, each on a random sample	One after another; each tree focuses on the previous trees' mistakes
Typical tree size	Deep (depths of 20+ are common)	Shallow (XGBoost usually peaks between depth 4 and 8)
Final answer	Average / vote of all trees	Sum of all trees' (scaled) outputs
Main risk	Needs many deep trees; less sharp	Can overfit if too many / too deep trees

1.4 Boosting: learn from your mistakes

Boosting works like a student who keeps a mistake notebook. First the model makes a rough guess. Then it looks at **how wrong** it was on every row (these errors are called **residuals**), and builds a small tree whose only job is to predict those errors. The corrections are added to the guess, new errors are computed, and the next tree tries to fix what is still wrong.

Trees are built ONE AFTER ANOTHER (sequentially). Each looks at the remaining errors.



final prediction = base guess + η·tree1 + η·tree2 + ... + η·treeN

η (eta) = learning_rate: how much of each tree's correction we accept

Figure 2 – Boosting is a chain. Every tree is trained on the errors left by all earlier trees.

A complete worked example you can follow by hand

Four houses; we predict the price (in thousands). Real prices: `y = [10, 14, 30, 34]`. Use learning rate `η = 0.5`. Each tree is a single split: small houses | big houses, and each leaf predicts the **average residual** of its rows.

Step	House 1	House 2	House 3	House 4	What happened
Real price y	10	14	30	34	what we want
Start: guess = mean	22	22	22	22	mean of y is 22
Residual = y − guess	−12	−8	+8	+12	errors to fix
Tree 1 leaf values	−10	−10	+10	+10	average residual of each group
New guess = 22 + 0.5·tree1	17	17	27	27	accept only 50% of the correction (η)
New residual	−7	−3	+3	+7	errors got smaller

Step	House 1	House 2	House 3	House 4	What happened
Tree 2 leaf values	−5	−5	+5	+5	average of the new residuals
New guess = prev + 0.5·tree2	14.5	14.5	29.5	29.5	closer again
New residual	−4.5	−0.5	+0.5	+4.5	keep going ...

TIP Why not accept 100% of each correction?

With $\eta = 1$ tree 1 would have jumped straight to 12 / 32. That feels great on the training rows, but it is exactly how the model starts memorising noise. A smaller step (`learning_rate`) means each tree only nudges the answer, so many different trees must agree before the prediction moves much. That is the heart of robust boosting.

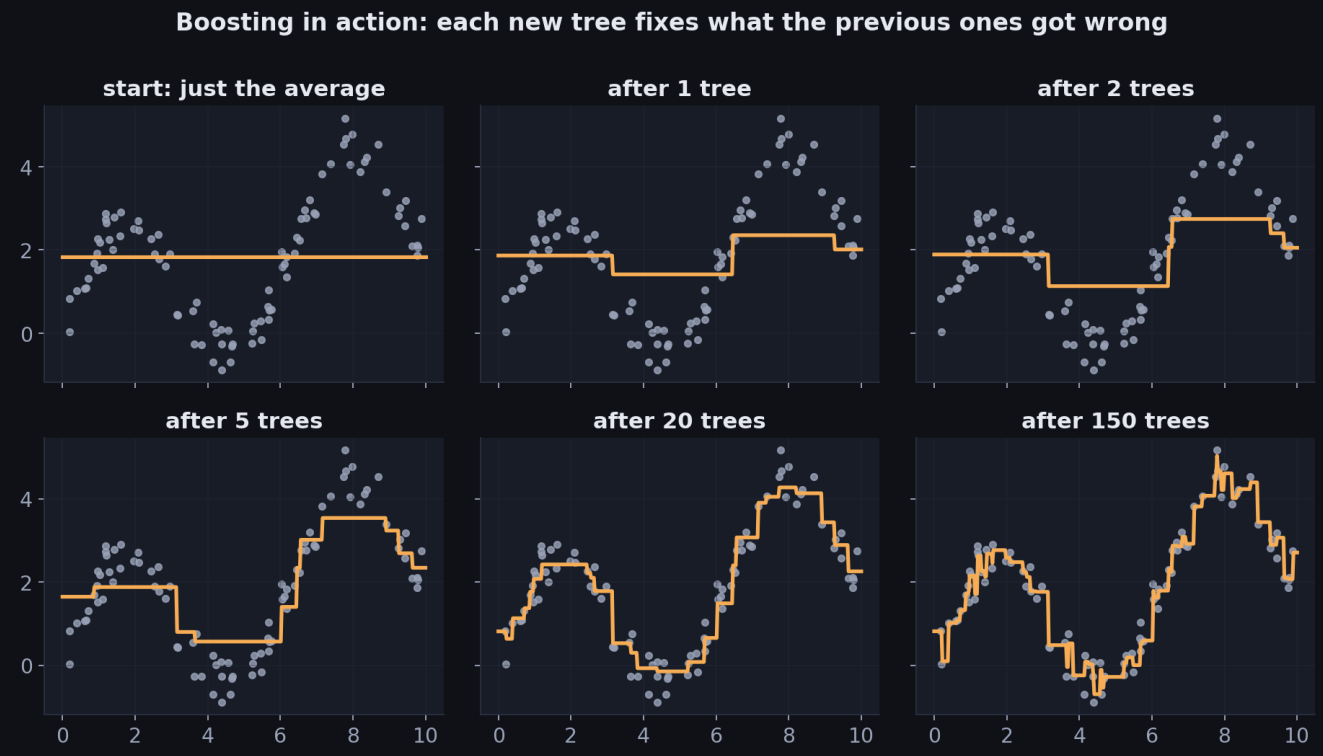


Figure 3 – Real XGBoost on a wavy 1-D dataset. The orange line is the prediction; watch it bend toward the data as trees are added.

1.5 So what is XGBoost?

XGBoost = eXtreme Gradient Boosting. It is boosting as described above, plus a set of upgrades that made it the favourite of data-science competitions:

- It uses a **regularised objective**: the model is penalised for complexity, so trees stay simple (Part 4 shows the maths).
- It uses **second-order information** (gradient and curvature) to choose splits and leaf values very accurately.
- It is **sparsity-aware**: missing values are handled natively.
- It is **engineered for speed**: histogram-based splits, multi-core and GPU support, cache-friendly data layout.
- It offers dozens of **hyperparameters** — the subject of this tutorial.

1.6 What is a hyperparameter?

A **parameter** is something the model learns from data (e.g. the leaf values and the split questions). A **hyperparameter** is a setting you choose before training that controls how the learning happens. Think of a car: the road decides where you go (parameters), while the dials and levers — gears, steering sensitivity, cruise-control — are yours (hyperparameters).

XGBoost hyperparameters act as the dials and levers that balance **how fast the model learns**, **how complex it becomes**, and **how well it generalises to unseen data**.

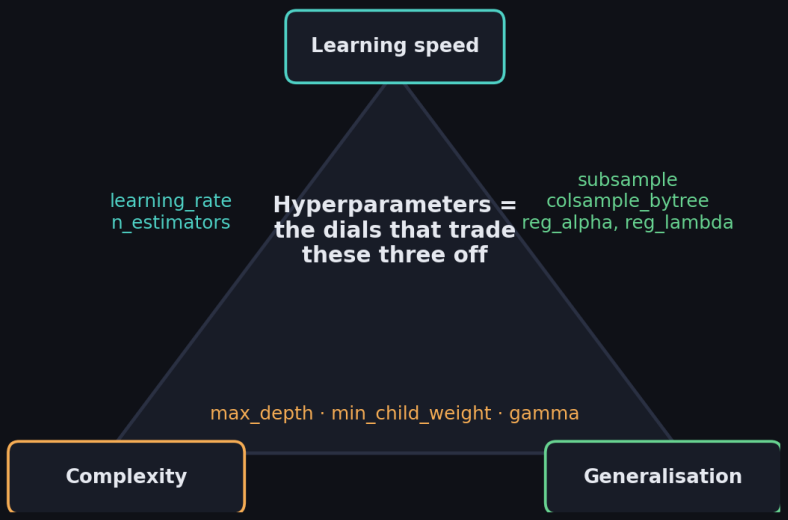


Figure 4 – Every hyperparameter is a dial on one of these three corners.

Underfitting vs overfitting — the two ways to fail

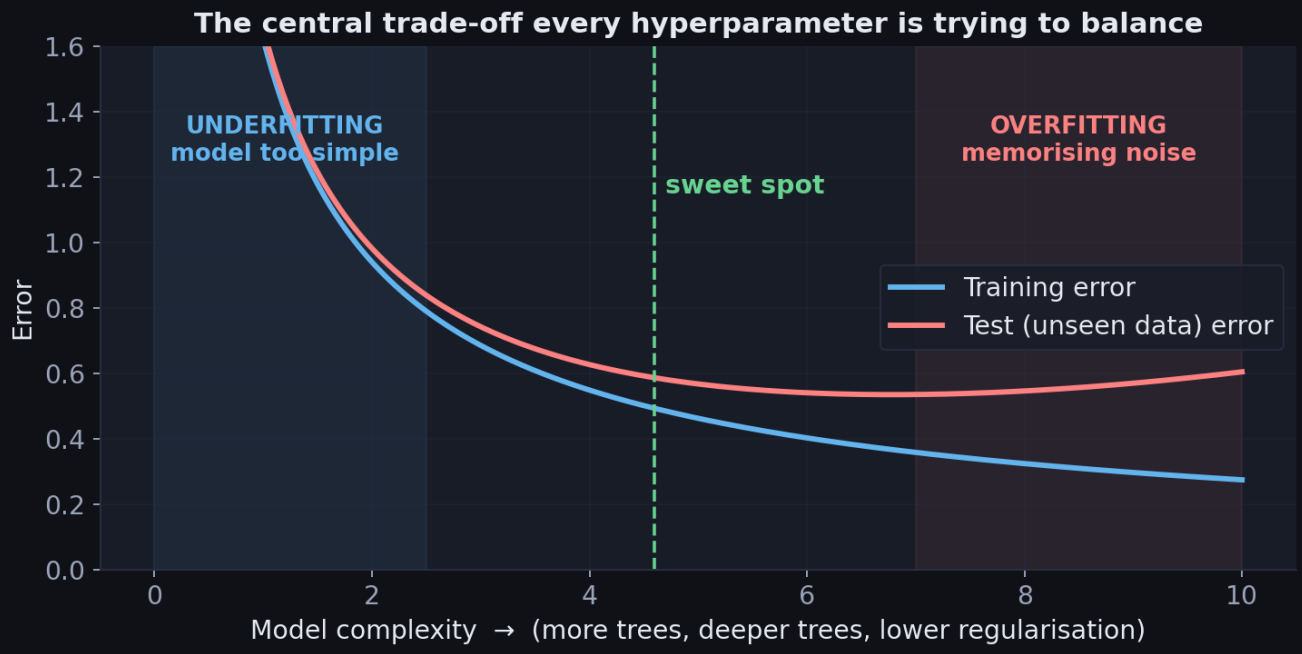


Figure 5 – As complexity grows, training error always falls, but test error falls and then rises again. The best model sits at the sweet spot.

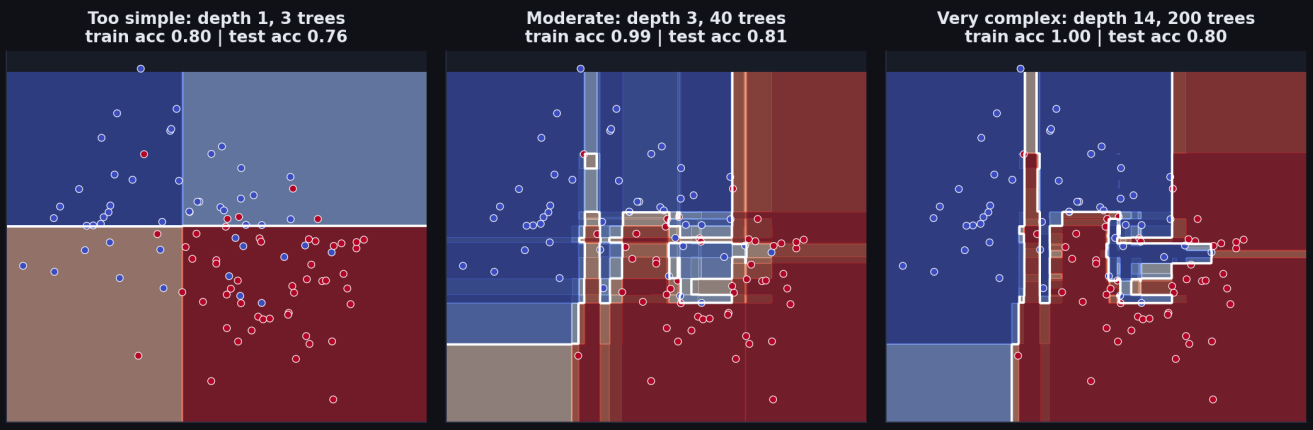


Figure 6 – The same data and the same algorithm with three complexity levels (accuracies measured on this tiny toy set, so treat them as an illustration). Left: a boundary that is too crude. Right: a jagged boundary that wraps around individual noisy points.

- **Underfitting (high bias):** the model is too simple to capture the real pattern. Fix: more trees, deeper trees, less regularisation.
- **Overfitting (high variance):** the model memorises noise. Fix: shallower trees, higher `min_child_weight`, more regularisation, sampling, lower learning rate with early stopping.

1.7 Your first XGBoost model (5 lines)

first_model.py

```
import xgboost as xgb
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = xgb.XGBClassifier()          # all defaults
model.fit(X_train, y_train)          # builds 100 trees, one after the other
print("accuracy:", model.score(X_test, y_test))
```

Everything you will learn next is about replacing the defaults in `XGBClassifier()` with better-chosen values.

PART 2 · The Dials (Level 2)

Now we meet every important hyperparameter. They fall into four families depending on what they control. Learn the family first; the individual names then become easy to remember.

Family	Parameter	One-line meaning	Default	Raise it →
1 Core learning	`n_estimators`	number of trees	100	more power, more overfit risk, slower
1 Core learning	`learning_rate (eta)`	step size / how much of each tree is trusted	0.3	faster learning, less robust
2 Tree complexity	`max_depth`	levels per tree	6	more complex trees
2 Tree complexity	`min_child_weight`	min data “weight” in a leaf	1	MORE conservative
2 Tree complexity	`gamma`	min gain needed to split	0	MORE conservative
3 Randomness	`subsample`	fraction of rows per tree	1.0	less randomness
3 Randomness	`colsample_bytree`	fraction of columns per tree	1.0	less randomness
4 Regularisation	`reg_alpha (L1)`	penalty on |weights| (sparsity)	0	MORE conservative
4 Regularisation	`reg_lambda (L2)`	penalty on weights² (smoothing)	1	MORE conservative



Figure 7 – The learning path of this tutorial.

2.1 Family 1 – Core learning parameters

These dictate the overall pace and length of the training process.

n_estimators — the number of trees

The total number of decision trees the model will build. More trees can learn more complex patterns, but they also increase the risk of overfitting and the training time. In the worked example of Part 1, each extra tree made the predictions better — but only up to a point.

learning_rate (eta) — the step size

The step size the model takes to correct the errors of previous trees. A smaller learning rate means the model learns more slowly and needs more trees (`n_estimators`), but it usually produces a more accurate and robust final model.



Figure 8 – Learning as walking down a bowl-shaped loss. Tiny steps are safe but slow; too-large steps jump over the bottom and can even diverge.

> **NOTE The two are a pair**
>
> Total progress ≈ learning_rate × number of trees. Halve the learning rate and you roughly need twice as many trees to get to the same place. Never tune one while forgetting the other.

2.2 Family 2 – Tree complexity (overfitting control)

These parameters stop individual trees from memorising the training data.

max_depth

How many levels deep a single tree can grow. Deeper trees capture highly specific, complex interactions in the data, but are the fastest way to cause overfitting. The number of possible leaves doubles with every level:

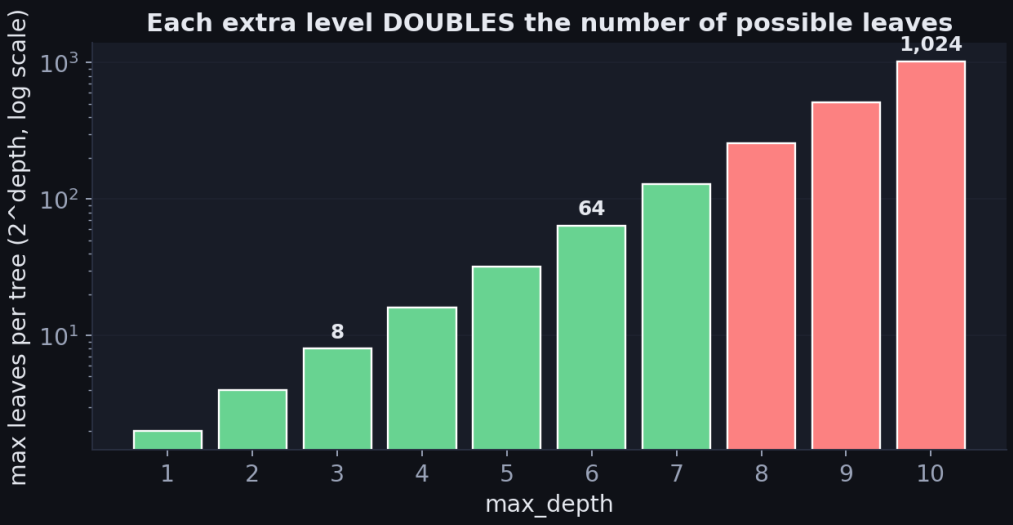


Figure 9 – Depth 3 allows 8 leaves; depth 10 allows 1,024 leaves — enough to isolate individual rows.

min_child_weight

The minimum amount of “weight” (roughly the number of samples) required in a new leaf. Higher values stop the tree from branching out for just one or two weird data points.

gamma (minimum split loss)

The minimum reduction in error required to justify making a new split. It acts as a strict gatekeeper: if a split does not improve the model by at least `gamma`, the tree stops growing there.

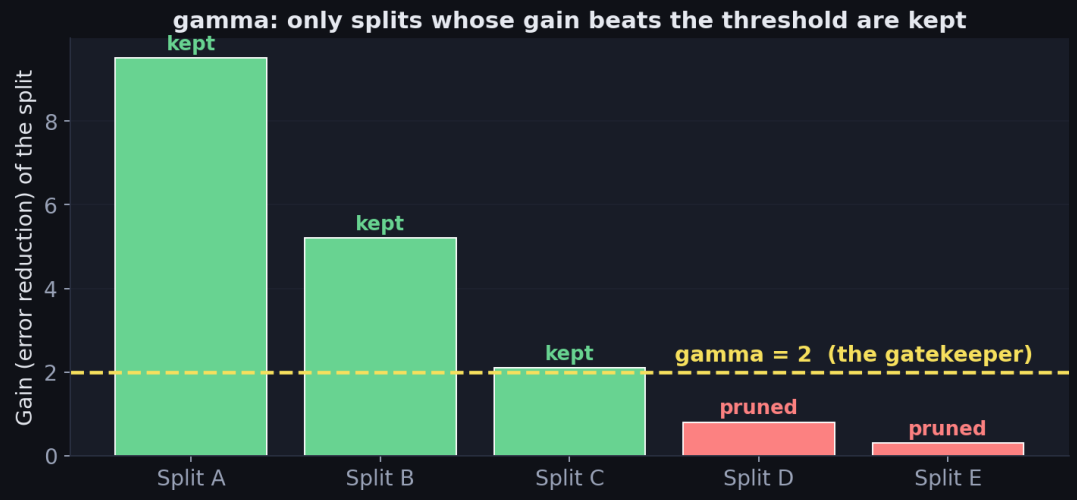


Figure 10 – Candidate splits and their gain. With gamma = 2, splits D and E are pruned away.

2.3 Family 3 – Randomness and sampling

Introducing randomness forces the model not to rely too heavily on any single feature or row of data.

subsample

The fraction of rows (data points) randomly chosen to build each tree. If set to 0.8, XGBoost randomly uses 80% of the training data before growing each tree.

colsample_bytree

The fraction of columns (features) randomly chosen to build each tree. Similar to a Random Forest, this prevents the model from relying solely on the single most predictive feature.

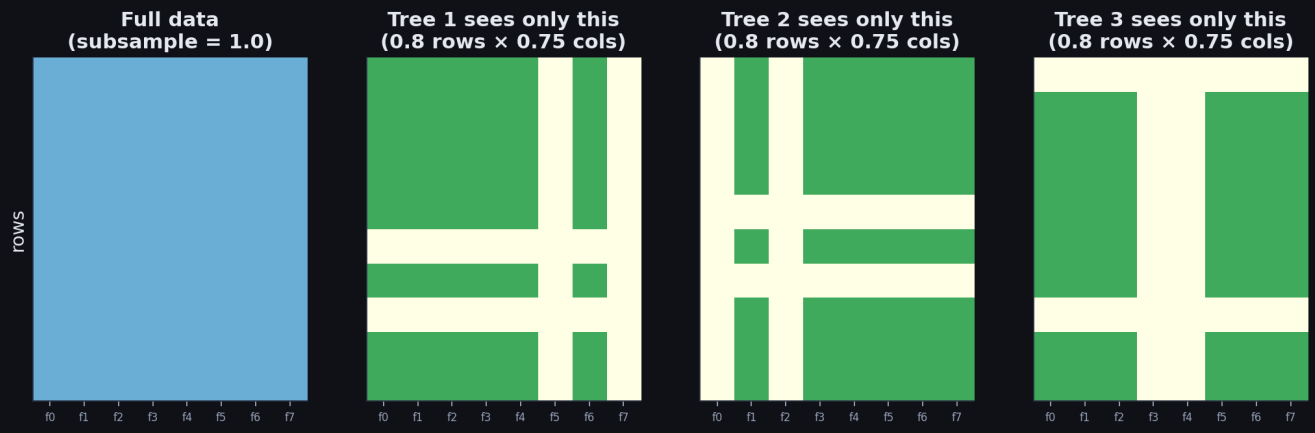


Figure 11 – Every tree sees a different random window of rows and columns (green = visible to that tree).

> **NOTE Related knobs**
>
> XGBoost also has `colsample_bylevel` and `colsample_bynode`, which re-sample columns at every depth level or every split. They multiply with `colsample_bytree`.

2.4 Family 4 – Regularisation (a penalty for complexity)

These act as a mathematical penalty for complexity: the model must "pay" for large leaf weights.

reg_alpha — L1 regularisation

Encourages weights to drop to **exactly zero**, effectively removing irrelevant contributions. Good for high-dimensional data (many features).

reg_lambda — L2 regularisation

Shrinks the weights **smoothly**, preventing any single feature or leaf from having an oversized influence on the prediction.

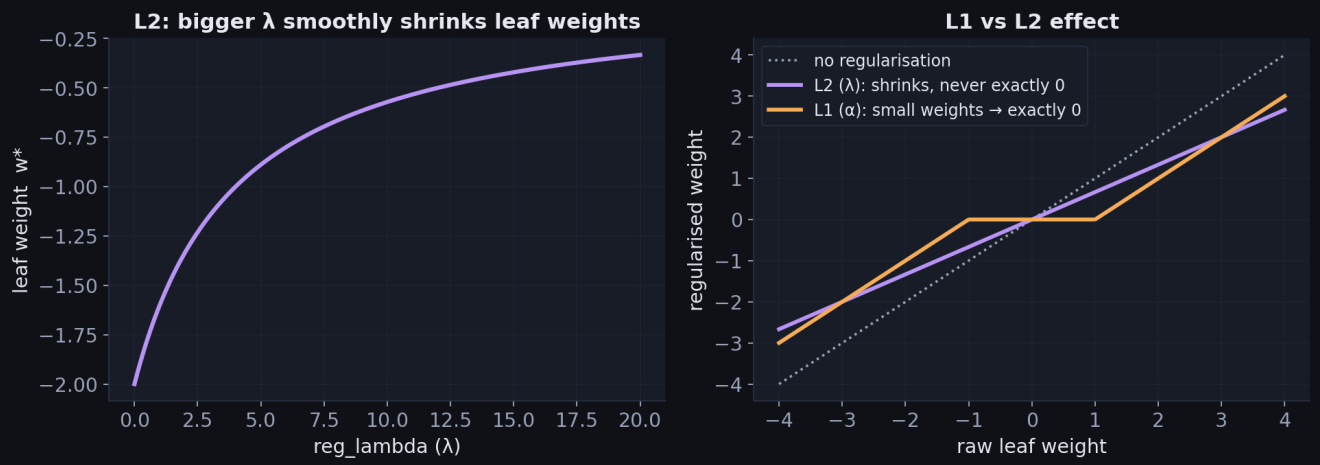


Figure 12 – Left: the optimal leaf weight shrinks as lambda grows. Right: L2 scales every weight down; L1 sets small weights to exactly zero ("soft-thresholding").

2.5 The Grid Search Cheat Sheet

Grid search means: list a few candidate values per hyperparameter, train a model for every combination, and keep the combination with the best cross-validated score. In scikit-learn this is `GridSearchCV` (all combinations) or `RandomizedSearchCV` (a random subset). You want a sensible range without a grid so massive that it takes weeks to compute.

Hyperparameter	Default	Standard grid values	Tuning strategy
`learning_rate`	0.3	`[0.01, 0.05, 0.1, 0.2]`	Lower is better, but requires higher n_estimators.
`n_estimators`	100	`[100, 300, 500, 1000]`	Tune alongside learning_rate.
`max_depth`	6	`[3, 5, 7, 9]`	Start small. Values over 10 rarely help.
`min_child_weight`	1	`[1, 3, 5, 7]`	Increase if the model is overfitting.
`gamma`	0	`[0, 0.1, 0.2, 0.3]`	Increase slowly to force conservative splits.
`subsample`	1.0	`[0.6, 0.8, 1.0]`	0.8 is a common sweet spot.
`colsample_bytree`	1.0	`[0.6, 0.8, 1.0]`	Lower this if you have hundreds of features.
`reg_alpha`	0	`[0, 0.01, 0.1, 1]`	Add to push useless features to 0.
`reg_lambda`	1	`[0.1, 1, 10]`	Increase to smooth out aggressive weights.

TIP **The 90% rule**

While XGBoost has many parameters, you only need to aggressively tune a handful of them to get about 90% of the possible performance gain. The most critical hyperparameters — the ones data scientists tune on almost every project — are `learning_rate`, `max_depth` and `min_child_weight`. Part 3 is a deep dive into exactly these three.

WATCH OUT **The size of the grid explodes**

The number of model fits is the **product** of the number of values per parameter, multiplied by the cross-validation folds. All nine parameters with the cheat-sheet values above would be $4^6 \times 3^3 = 110{,}592$ combinations — before multiplying by the cross-validation folds. Part 5 shows how to avoid this.

PART 3 · The “Big Three” Deep Dive (Level 2+)

If you only have time to tune three things, tune these. For each one we use the same template: **how it works → how it helps → how it affects the model → tuning strategy → grid values**. The figures in this part come from real XGBoost runs on a noisy 2,500-row toy dataset, so you can see the effects, not just read about them.

3.1 learning_rate (eta) and n_estimators

These two parameters are inextricably linked. You cannot tune one without understanding the other.

How it works

XGBoost builds trees sequentially. Each new tree looks at the errors (residuals) made by all the previous trees combined, and tries to predict and correct those errors. The `learning_rate` is a multiplier applied to the predictions of each new tree.

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(x)$$

new prediction = old prediction + learning rate × (what tree t says)

How it helps

If the learning rate is 1.0, the model accepts the new tree's corrections completely. If it is 0.1, the model only accepts 10% of the correction. This forces the model to learn slowly and incrementally.

How it affects the model

- **High learning rate (e.g. 0.3+):** the model learns very fast and needs fewer trees. However it takes massive steps and can easily overshoot the optimal solution, leading to a brittle model that overfits.
- **Low learning rate (e.g. 0.01):** the model takes tiny steps. It becomes highly robust and generalises well to new data. The catch? It requires a massive number of trees to reach the finish line, which takes much longer to train.

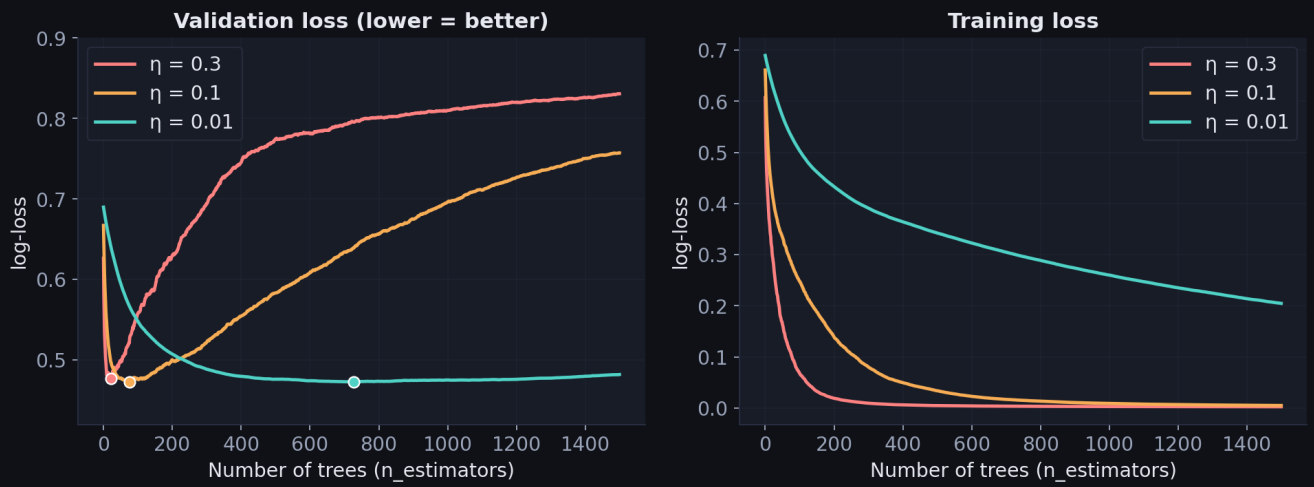


Figure 13 – Real run. Dots mark each curve's best validation round: η=0.3 peaks after only 21 trees and then overfits quickly; η=0.1 peaks at 75; η=0.01 needs about 727 trees but ends with the flattest, most stable curve.

The tuning strategy

You generally don't grid-search these simultaneously. The best practice: **pick a fixed, low learning rate, then use early stopping to find the exact right number of trees** (details in Part 5).

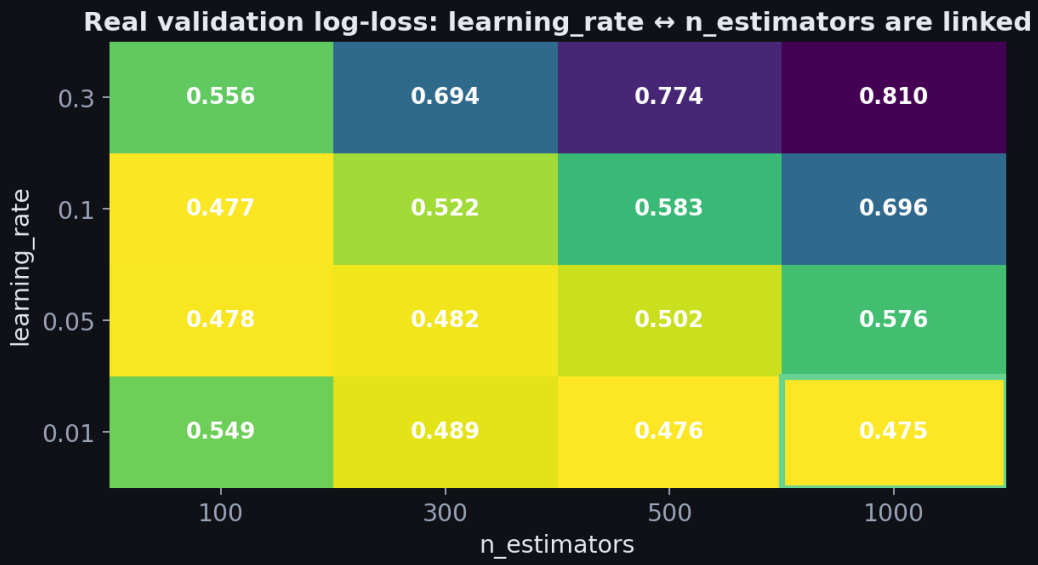


Figure 14 – Validation log-loss for every learning_rate × n_estimators pair (lower/yellow is better; green box = best). Notice the diagonal: a smaller rate needs more trees, a large rate with many trees is a disaster.

Grid search values

Parameter	Values
learning_rate	[0.01, 0.05, 0.1]
n_estimators	[100, 500, 1000]

3.2 max_depth

This is the primary lever for controlling how complex your model is allowed to become.

How it works

It determines the maximum number of nodes from the root (top) of a tree down to the furthest leaf (bottom). A depth of 1 is a “stump” (one split). A depth of 3 means three levels of splits (like Figure 1).

How it helps

Data often has complex, multi-variable interactions (for example “If Age > 40 AND Income > 50k AND Location = City”). Deeper trees can learn these specific interactions.

How it affects the model

- **Too deep (e.g. 10+):** the tree becomes so hyper-specific that it starts memorising the noise in your training data. It performs flawlessly on the training set but crashes on the test set (severe overfitting).
- **Too shallow (e.g. 2):** the model cannot capture the complex patterns in the data (underfitting).



Figure 15 – Real experiment (150 trees, η=0.1). Training loss keeps falling toward zero, while test loss is best at max_depth = 5 and then drifts upward.

Grid search values

Start small: `max_depth: [3, 5, 7, 9]`. Note: unlike Random Forests, which often use depths of 20+, XGBoost usually peaks between 4 and 8. This is because boosting adds many trees together, so each tree only needs to be a modest, shallow “correction”.

3.3 min_child_weight

If `max_depth` stops trees from getting too **tall**, `min_child_weight` stops them from getting too **thin** or specific at the leaf level.

How it works

In XGBoost, “weight” essentially refers to the sum of the data points (specifically the sum of the instance weights or second-order derivatives, called the Hessian) inside a node. In plain English: it is roughly the **minimum number of data samples** required to allow a node to exist. (The exact meaning is in Part 4.)

How it helps

Imagine a tree splitting the data perfectly until it isolates one single person in your dataset who happens to have a very unusual outcome. That is terrible for generalisation. `min_child_weight` acts as a **bouncer**: if a proposed split would result in a leaf with too few data points, the split is rejected.

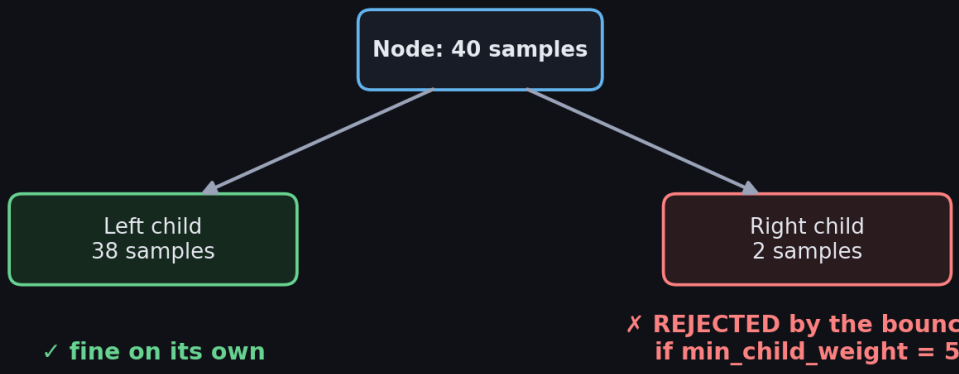


Figure 16 – The bouncer in action. The left child is big enough, but the right child holds only 2 samples, so the whole split is refused.

How it affects the model

- **Low value (e.g. 1):** the model is allowed to create highly specific leaf nodes. This increases the risk of overfitting to outliers.
- **High value (e.g. 10+):** the model is forced to be more conservative. It only creates splits that represent a larger, more significant portion of the data.

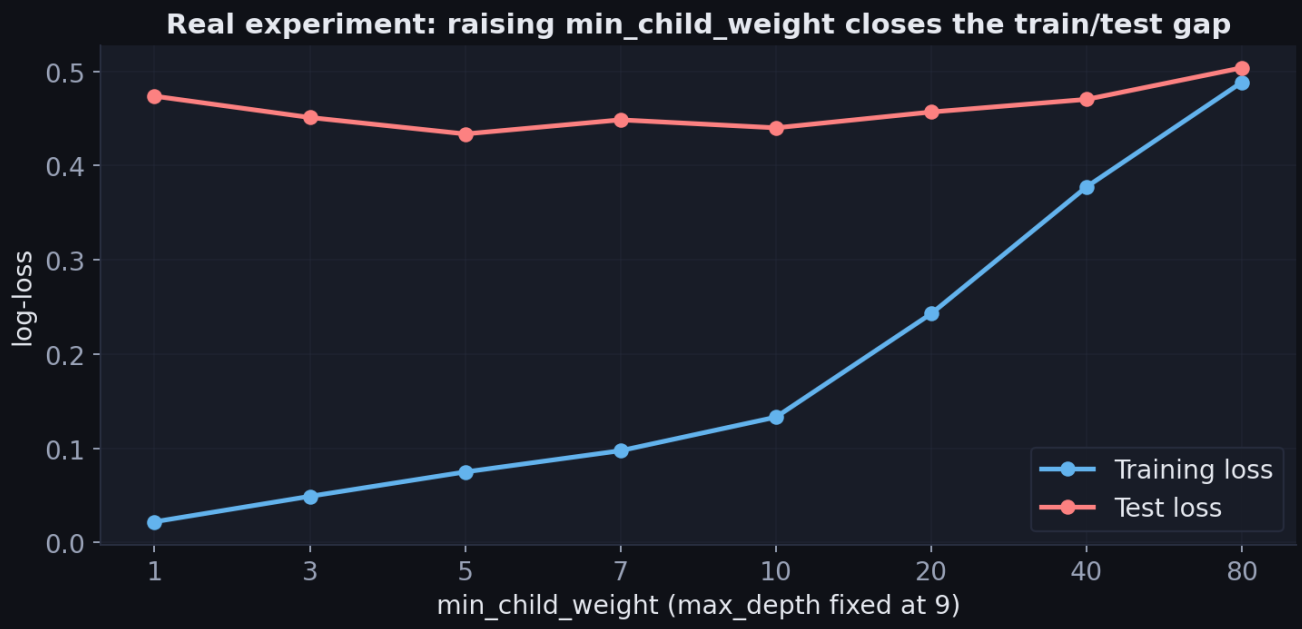


Figure 17 – Real experiment with very deep trees (max_depth = 9). As min_child_weight rises the training loss climbs (less memorising) and the gap to test loss closes. Best test loss here: min_child_weight = 5.

Grid search values

Parameter	Values
`min_child_weight`	`[1, 3, 5, 7]`

3.4 The “Second Tier”: subsample and colsample_bytree

If you have tuned the big three and your model is still overfitting, the next parameters to turn to are the sampling parameters. These add randomness.

How they work

- `subsample`: randomly selects a fraction of the rows (observations) before growing a tree.
- `colsample_bytree`: randomly selects a fraction of the columns (features) before growing a tree.

How it helps

It prevents the model from relying too heavily on any single dominant feature or specific subset of training data. It forces the model to look at the problem from different angles — just as you get a better opinion by asking several people who each saw different evidence.

Grid search values

Parameter	Values
`subsample`	`[0.7, 0.8, 0.9]`
`colsample_bytree`	`[0.7, 0.8, 0.9]`

3.5 How the big three interact

If you see...	Likely cause	First dial to try
Train loss ≪ test loss	trees too specific	lower max_depth, raise min_child_weight
Both losses high	model too simple / too few trees	raise max_depth, more n_estimators
Validation loss rises while trees are added	too many trees for this learning rate	early stopping, or lower learning_rate
Training takes forever	low rate × many trees, exact splits	raise rate to ≈0.1, tree_method='hist'
Still overfit after the big three	model leans on a few rows/features	subsample / colsample_bytree 0.7–0.9, then reg_lambda

PART 4 · How It Works Mathematically (Level 3)

This part opens the hood. Everything here is optional if you only want to tune models — but it makes every hyperparameter **logical** instead of magical. Take it slowly; each step is one small idea. Maths terms are always followed by plain English.

4.1 Notation cheat-sheet

Symbol	Read as	Meaning
x_i, y_i	“x-i, y-i”	features and true target of row i
$\hat{y}_i$	“y-hat”	model prediction for row i
f_k		the k-th tree (a function that returns a leaf weight)
K or t		number of trees / index of the current tree
w_j		leaf weight (score) of leaf j
T		number of leaves in a tree
$l(y, \hat{y})$	“loss”	how wrong one prediction is
$\Omega(f)$	“omega”	complexity penalty of a tree
η	“eta”	learning_rate
λ, α, γ	lambda, alpha, gamma	reg_lambda, reg_alpha, gamma
g_i, h_i		first and second derivative of the loss for row i (“gradient”, “hessian”)
G_j, H_j		sums of g and h over all rows in leaf j

4.2 The model is a sum of trees

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i)$$

the final prediction is the sum of what every tree outputs for row i

Each tree f_k maps a row to one of its leaves, and the leaf holds a number. With the learning rate included, we add trees one at a time:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$$

4.3 What are we minimising? The objective

$$\text{Obj} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

total error on the training data + a penalty for complicated trees

The first term says **fit the data**. The second says **stay simple**. The tension between them is exactly the underfitting/overfitting trade-off of Figure 5. For one tree, XGBoost defines the complexity penalty as:

$$\Omega(f) = \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j|$$

γT: price per leaf | ½λΣw²: L2 penalty (reg_lambda) | αΣ|w|: L1 penalty (reg_alpha)

MATH Where your hyperparameters live

gamma charges a fixed price for every extra leaf (so fewer leaves). reg_lambda punishes big leaf weights quadratically (smooth shrinkage). reg_alpha punishes them linearly (pushes small ones to exactly 0). That is all they do.

4.4 Training one tree at a time (additive training)

We cannot optimise all trees at once, so at round t we keep the old trees fixed and add one new tree `f_t` that minimises:

$$\text{Obj}^{(t)} = \sum_{i=1}^{n} l\left(y_i,\ \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

4.5 The Taylor trick — why XGBoost is “extreme”

The true loss can be complicated. XGBoost replaces it, around the current prediction, with a simple parabola using the first derivative (**gradient** g) and second derivative (**hessian** h, the curvature):

$$\text{Obj}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \tfrac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

(constant terms that do not depend on f_t were dropped)

$$g_i = \frac{\partial\, l(y_i, \hat{y}_i^{(t-1)})}{\partial\, \hat{y}_i^{(t-1)}} \qquad h_i = \frac{\partial^2\, l(y_i, \hat{y}_i^{(t-1)})}{\partial\, (\hat{y}_i^{(t-1)})^2}$$

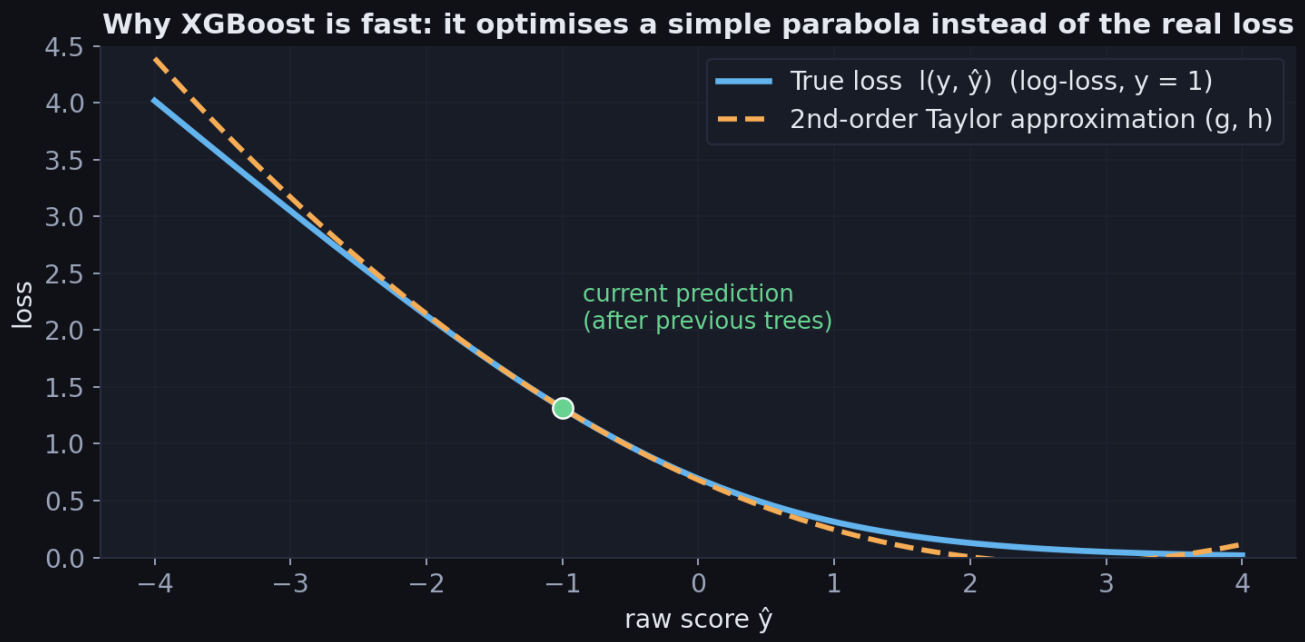


Figure 18 – Near the current prediction the real loss (blue) and its quadratic approximation (orange) are almost identical. Minimising a parabola is easy and has a closed-form answer.

4.6 Optimal leaf weight

All rows that land in the same leaf j get the same output w_j. Group the sums by leaf, and define $G_j = \Sigma\, g_i$ and $H_j = \Sigma\, h_i$ over the rows in that leaf:

$$\text{Obj}^{(t)} = \sum_{j=1}^{T} \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T$$

For each leaf this is a simple parabola in w_j. Setting its derivative to zero gives the best weight:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

the optimal leaf value = − (sum of gradients) / (sum of hessians + lambda)

ANALOGY Why lambda shrinks

λ sits in the denominator. Big λ → bigger denominator → leaf weight closer to 0 → the leaf makes a more cautious correction. With λ = 0 and squared-error loss, the formula reduces to “average residual of the rows in the leaf” — exactly what we computed by hand in Part 1.

4.7 Choosing splits: the Gain formula

Plugging w* back in gives the score of a tree structure. The **improvement from splitting** one node into a left (L) and right (R) child is:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

score of the two children − score of the unsplit parent − price of one extra leaf

- **Gain > 0:** the split improves the objective → keep it.
- **Gain ≤ 0 (because of γ):** pruned. This is exactly the `gamma` gatekeeper of Figure 10.
- **H_L or H_R below min_child_weight:** split is not even considered. This is the bouncer of Figure 16.

Worked example with real numbers

Reuse the four houses from Part 1 with squared-error loss, `l = ½(y − ŷ)²`, so $g_i = \hat{y}_i - y_i$ = −residual and $h_i = 1$. Start guess 22, residuals [−12, −8, +8, +12], so g = [+12, +8, −8, −12] and h = [1, 1, 1, 1]. Parent: G = 0, H = 4.

Candidate split	G_L, H_L	G_R, H_R	Gain (λ=0, γ=0)	Gain (λ=1, γ=0)
after house 1	12, 1	−12, 3	½[144/1 + 144/3 − 0] = 96	½[72 + 36 − 0] = 54
after house 2 (middle)	20, 2	−20, 2	½[400/2 + 400/2 − 0] = 200	½[133.3 + 133.3 − 0] = 133.3
after house 3	12, 3	−12, 1	96	54

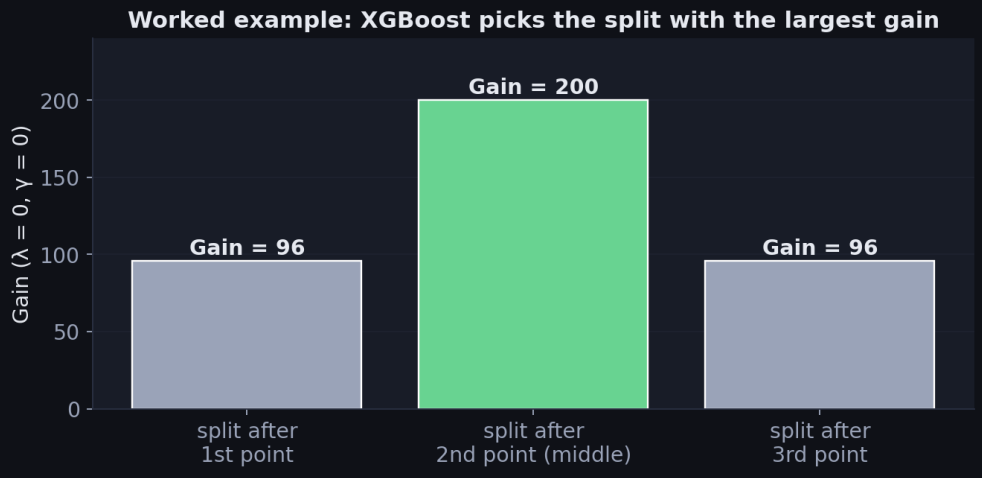


Figure 19 – The middle split wins. Leaf weights for it: w = −G/(H+λ) → left −20/2 = −10 (λ=0), right +10. With λ=1 they shrink to −6.67 and +6.67.

4.8 Squared error vs logistic loss: what g and h look like

Task	Loss l(y, ŷ)	g	h
Regression (reg:squarederror)	½ (y − ŷ)²	ŷ − y	1
Binary classification (binary:logistic)	−[y·ln p + (1−y)·ln(1−p)], p = 1/(1+$e^{-\hat{y}}$)	p − y	p (1 − p)

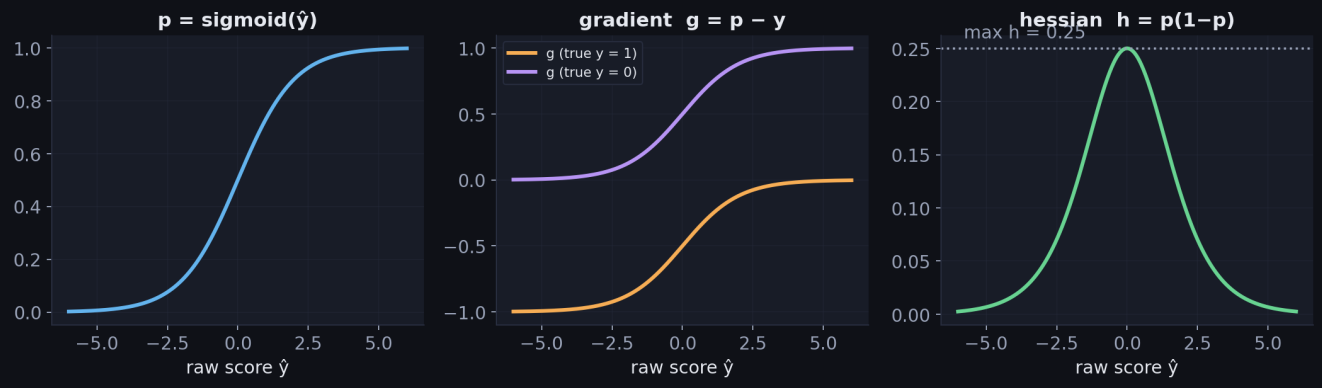


Figure 20 – For classification h = p(1−p) never exceeds 0.25 and is tiny when the model is already confident (p near 0 or 1).

> **WATCH OUT** **min_child_weight is not “number of samples” for classification**
>
> min_child_weight is the **sum of hessians** H in a leaf. For squared error h = 1, so it equals the sample count. For logistic loss h ≤ 0.25, so a leaf with min_child_weight = 1 needs at least 4 samples, and many more when the model is confident. This is why the “roughly the number of samples” rule is only an approximation.

4.9 Where each hyperparameter appears in the maths

Hyperparameter	Appears in	Effect on the formulas
learning_rate η	$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta\, f_t$	scales every tree's contribution
n_estimators	K (number of terms in the sum)	more terms → more flexible
max_depth	structure of f_t	$T \le 2^{\text{depth}}$ leaves
min_child_weight	H_L, $H_R \ge$ threshold	filters candidate splits
gamma γ	Ω: γT and Gain − γ	charge per leaf; minimum gain
reg_lambda λ	$H_j + \lambda$	shrinks w*, lowers gain of tiny splits
reg_alpha α	α|w| in Ω	soft-threshold small leaf weights to 0
subsample	which rows i enter the sums	random subset of rows per tree
colsample_*	which features are split candidates	random subset of columns per tree

4.10 Finding the best split: three algorithms

- **Exact greedy:** sort each feature, sweep through every value, compute Gain at each cut. Perfectly accurate, but slow on big data.
- **Approximate / histogram (tree_method='hist'):** first bucket each feature into ≤256 bins, then only test bin borders. Almost the same accuracy, far fewer candidates (Part 5).
- **Sparsity-aware:** for missing values, compute Gain with them sent left and right; keep the better default direction.

4.11 Build XGBoost from scratch (60 lines, verified)

To prove there is no magic, here is a complete squared-error XGBoost in plain NumPy. It uses exactly the formulas above: `g = pred − y`, `h = 1`, the Gain formula with λ, γ and min_child_weight, and the leaf weight −G/(H+λ).

tiny_xgb.py

```python
import numpy as np

def build_tree(X, g, h, depth, max_depth, lam, gamma, mcw):
    G, H = g.sum(), h.sum()
    best = None                                  # (gain, feature, threshold)
    if depth < max_depth:
        for f in range(X.shape[1]):              # try every feature ...
            order = np.argsort(X[:, f]); xs = X[order, f]
            GL_all, HL_all = np.cumsum(g[order]), np.cumsum(h[order])
            for i in range(len(xs) - 1):         # ... and every cut point
                if xs[i] == xs[i + 1]:
                    continue
                GL, HL = GL_all[i], HL_all[i]
                GR, HR = G - GL, H - HL
                if HL < mcw or HR < mcw:         # min_child_weight "bouncer"
                    continue
                gain = 0.5 * (GL**2/(HL+lam) + GR**2/(HR+lam) - G**2/(H+lam)) - gamma
                if gain > 0 and (best is None or gain > best[0]):
                    best = (gain, f, (xs[i] + xs[i + 1]) / 2)
    if best is None:                             # no good split -> make a leaf
        return {"leaf": -G / (H + lam)}          # optimal leaf weight  w* = -G/(H+lambda)
    _, f, thr = best
    m = X[:, f] < thr
    return {"f": f, "thr": thr,
            "left":  build_tree(X[m],  g[m],  h[m],  depth+1, max_depth, lam, gamma, mcw),
            "right": build_tree(X[~m], g[~m], h[~m], depth+1, max_depth, lam, gamma, mcw)}

def predict_one(tree, x):
    while "leaf" not in tree:
        tree = tree["left"] if x[tree["f"]] < tree["thr"] else tree["right"]
    return tree["leaf"]

class TinyXGB:                                   # squared-error regression
    def __init__(self, n_estimators=50, eta=0.3, max_depth=3, lam=1.0, gamma=0.0, mcw=1.0):
        self.n, self.eta, self.md, self.lam, self.gamma, self.mcw = n_estimators, eta, max_depth, lam, gamma, mcw
    def fit(self, X, y):
        self.base = y.mean(); pred = np.full(len(y), self.base); self.trees = []
        for _ in range(self.n):
            g = pred - y                         # gradient of 1/2 (y - pred)^2
            h = np.ones_like(y)                  # hessian is 1 for squared error
            t = build_tree(X, g, h, 0, self.md, self.lam, self.gamma, self.mcw)
            self.trees.append(t)
            pred += self.eta * np.array([predict_one(t, x) for x in X])   # eta * f_t(x)
        return self
    def predict(self, X):
        out = np.full(len(X), self.base)
        for t in self.trees:
            out += self.eta * np.array([predict_one(t, x) for x in X])
        return out
```

Check against the real library:

test_scratch.py

```python
import numpy as np, xgboost as xgb
from scratch import TinyXGB
rng=np.random.RandomState(0); X=rng.uniform(0,10,(300,2)); y=2*np.sin(X[:,0])+0.3*X[:,1]+rng.normal(0,.3,300)
mine=TinyXGB(30,0.3,3,1.0,0.0,1.0).fit(X,y).predict(X)
ref=xgb.XGBRegressor(n_estimators=30,learning_rate=0.3,max_depth=3,reg_lambda=1.0,gamma=0,min_child_weight=1,tree_method="exact",base_score=float(y.mean()),random_state=0).fit(X,y).predict(X)
print("max abs diff vs real XGBoost:",np.abs(mine-ref).max(), "rmse mine",np.sqrt(((mine-y)**2).mean()))
```

> **TIP Result of running this**
>
> On a 300-row test problem the predictions of this ~60-line implementation differ from real `XGBRegressor(tree_method='exact')` by at most **1.4×10^{-6}** — floating-point noise. Everything in this part is literally how XGBoost trains.

PART 5 · Pro Level: Speed & Efficiency

Getting maximum efficiency from XGBoost — both in training speed and predictive accuracy — requires moving beyond basic hyperparameter tuning and optimising how the algorithm interacts with your data and hardware. If your model takes too long to train you cannot iterate quickly, and iteration speed is what wins projects.

5.1 Speed and compute efficiency

Switch to the histogram method — tree_method='hist'

By default (older versions) XGBoost can use an **exact greedy algorithm** to find tree splits, which scans every single data point. For datasets over about 10,000 rows this is agonisingly slow. Setting `tree_method='hist'` bins your continuous features into discrete buckets (histograms). It yields nearly identical accuracy but can train **10× to 50× faster** on large data.

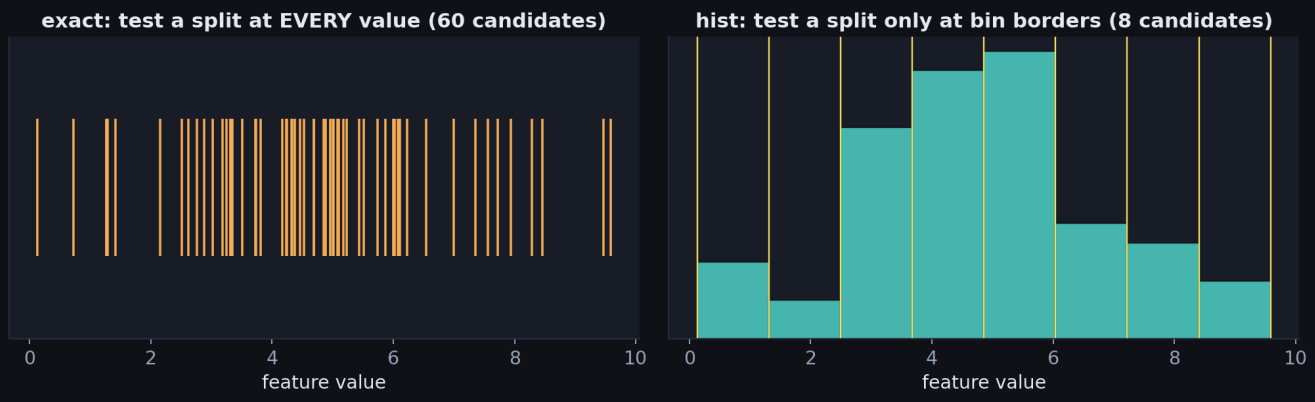


Figure 21 – Exact: one candidate split per value. Histogram: one candidate per bin border. Fewer candidates, almost the same best split.

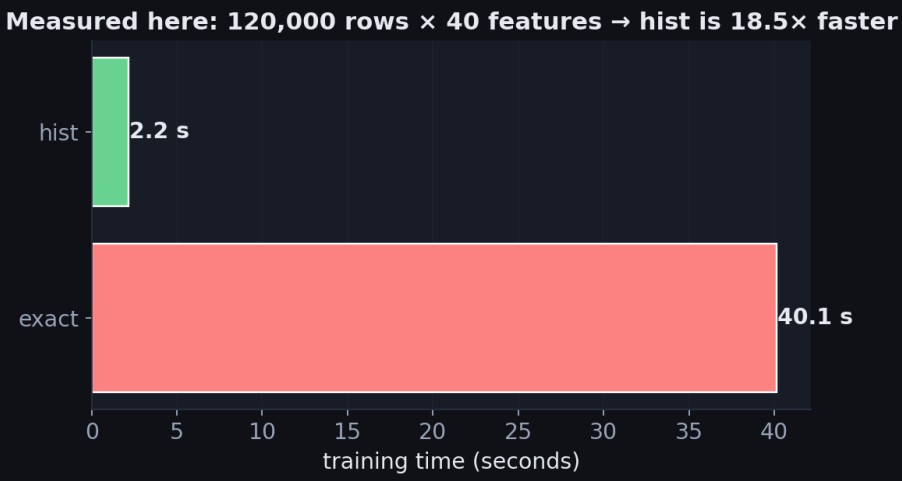


Figure 22 – Measured in this tutorial's sandbox (60 trees, depth 6): exact took 40.1 s, hist took 2.2 s. Your speed-up depends on data size and CPU.

> **NOTE Version note**
>
> In XGBoost 2.0 and later `tree_method='hist'` is already the default, so you mostly get this for free; setting it explicitly is still good practice because it documents intent and protects you when running older versions.

Leverage your GPU

If you are running XGBoost on a machine with an NVIDIA GPU (like in Google Colab or Kaggle), move the work onto the GPU. The matrix-heavy computation shifts to the GPU, cutting massive training jobs from hours to minutes. Older tutorials write `tree_method='gpu_hist'`; in XGBoost 2.0+ that name is deprecated and the modern equivalent is:

```
# XGBoost >= 2.0 (recommended)
model = xgb.XGBClassifier(tree_method="hist", device="cuda")

# XGBoost 1.x (older syntax from many tutorials)
model = xgb.XGBClassifier(tree_method="gpu_hist")
```

Always use early stopping

Never guess the optimal `n_estimators`. Set it artificially high (e.g. 2000), pick a low `learning_rate` (e.g. 0.05), and use `early_stopping_rounds=50`. The model monitors a validation set and automatically stops when performance has not improved for 50 trees. This saves immense compute time and prevents overfitting.

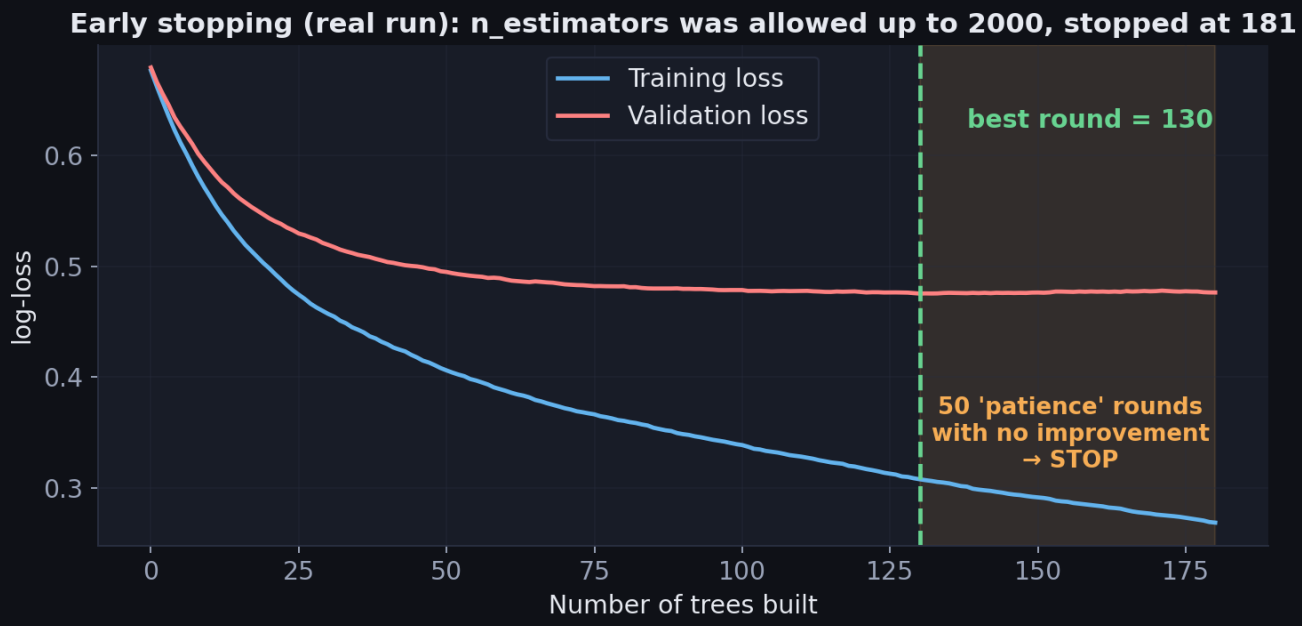


Figure 23 – Real run with η=0.05. Validation loss was best at tree 130; after 50 more trees without improvement training stopped at tree 181 instead of building all 2000. XGBoost then keeps the model from the best round.

early_stopping.py

```
import xgboost as xgb
model = xgb.XGBClassifier(
    n_estimators=2000,            # artificially high on purpose
    learning_rate=0.05,
    early_stopping_rounds=50,     # stop after 50 rounds without improvement
    eval_metric="logloss",
    tree_method="hist",
)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],   # a SEPARATE validation set
          verbose=False)
print("best number of trees:", model.best_iteration)
```

COMMON MISTAKE Do not early-stop on the test set

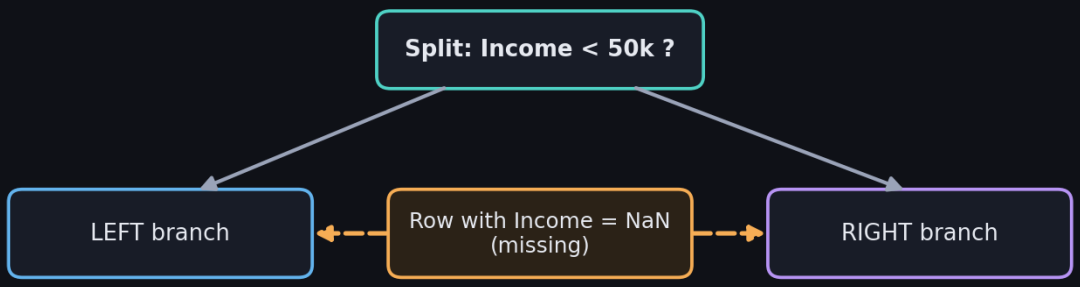
Use a separate validation split (or cross-validation) for early stopping, and keep the test set untouched until the very end. Otherwise your final score is optimistic.

5.2 Data and pipeline efficiency

XGBoost is smart enough to handle certain data transformations internally, saving you from writing slow, complex pre-processing pipelines.

Stop imputing missing values blindly

XGBoost has a built-in **sparsity-aware split-finding algorithm**. It automatically learns the best direction (left or right branch) to send missing values so as to minimise loss. Imputing with the mean or median artificially alters your data distribution and often hurts XGBoost's performance. Just feed it the NaN values directly.



XGBoost tries sending ALL missing rows left, then right

and keeps the direction with the better gain → it LEARNS a default direction per split

No mean/median imputation needed.

Figure 24 – A missing value is not an error: XGBoost tries both directions and remembers the winner for every split.

Use scale_pos_weight for imbalanced data

If you are predicting a rare event (e.g. 1% fraud, 99% legitimate), do not bloat your dataset by using SMOTE or oversampling. Instead, calculate the ratio of negative to positive samples and pass it to `scale_pos_weight`. This forces the algorithm to pay more attention to the minority class with zero computational overhead.

$$\text{scale_pos_weight} = \frac{\#\text{ negative samples}}{\#\text{ positive samples}}$$

e.g. 20,740 legitimate / 260 fraud ≈ 80

```
ratio = (y_train == 0).sum() / (y_train == 1).sum()
model = xgb.XGBClassifier(scale_pos_weight=ratio, tree_method="hist")
```

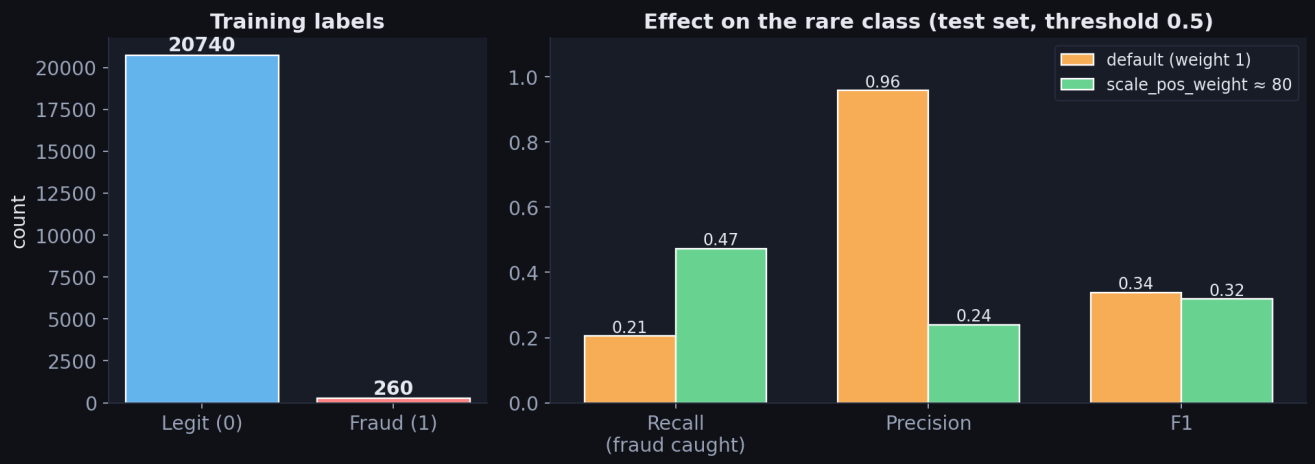


Figure 25 – Real experiment with 1% positives. With scale_pos_weight ≈ 80, recall (fraud caught) rises from 0.21 to 0.47, at the price of lower precision (0.96 → 0.24).

WATCH OUT It is a trade-off, not free accuracy

Reweighting moves the balance between “catch more positives” and “raise more false alarms”. F1 stayed about the same in this run. Choose by business cost, and consider tuning the decision threshold or using AUC-PR as the metric. Also remember: reweighting distorts predicted probabilities, so calibrate them if you need true probabilities.

Drop zero-importance features early

Run a quick baseline model and plot the feature importances (`xgb.plot_importance()`). Drop the bottom 20% of features that contribute nothing to the model. Fewer columns mean faster memory loading, faster split finding, and often better accuracy by removing noise.

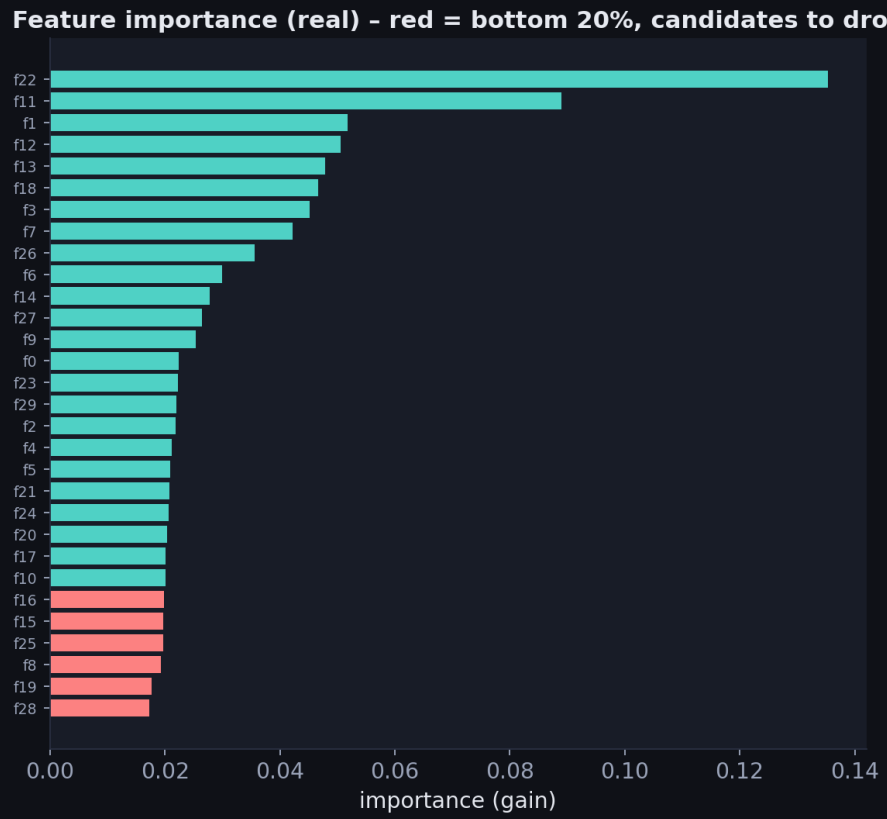


Figure 26 – Real feature importance (gain) for 30 features; the bottom 6 (red) are the candidates to drop.

NOTE What happened in the experiment

Training on all 30 features took 0.21 s (test log-loss 0.436); on the 24 kept features 0.17 s (0.447). On this small toy set, speed improved and accuracy changed only slightly (it did not improve). Pruning is most valuable with hundreds of columns; always confirm with validation data after dropping.

5.3 Tuning workflow efficiency

Tune in stages, not massive grids

Do not throw 6 parameters with 4 values each into a single GridSearchCV. That is 4^6 = 4,096 model fits (and ×3 or ×5 for cross-validation). Instead:

1. **First,** fix a learning rate and find the optimal number of trees (early stopping).
2. **Next,** tune `max_depth` and `min_child_weight` together.
3. **Then,** tune the sampling parameters (`subsample`, `colsample_bytree`).
4. **Finally,** lower the learning rate and run one last time with early stopping.

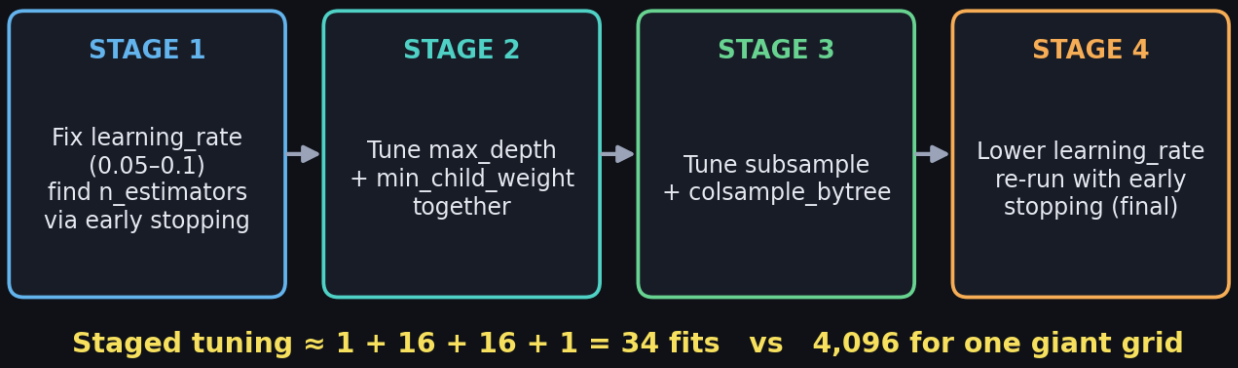


Figure 27 – The four-stage workflow.

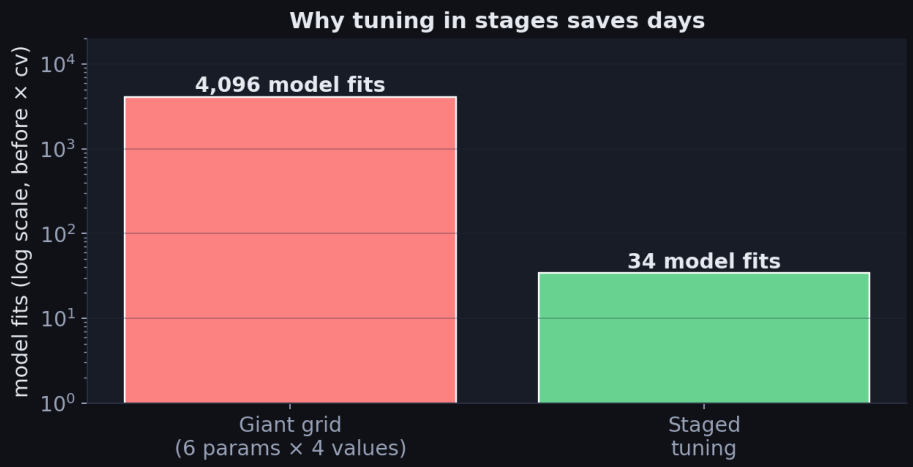


Figure 28 – About 34 fits versus 4,096. Same ingredients, ~120× less work (the staged search can miss rare interactions between stages, but in practice it gets very close).

Use RandomizedSearchCV or Optuna

For complex datasets, grid search is highly inefficient: it wastes trials re-testing the same values of the important parameter while varying unimportant ones. **Random search** tries different values every time. Bayesian-optimisation libraries like **Optuna** actively learn from previous tuning runs to guess the next best set of parameters, often finding a better model in a fraction of the time.

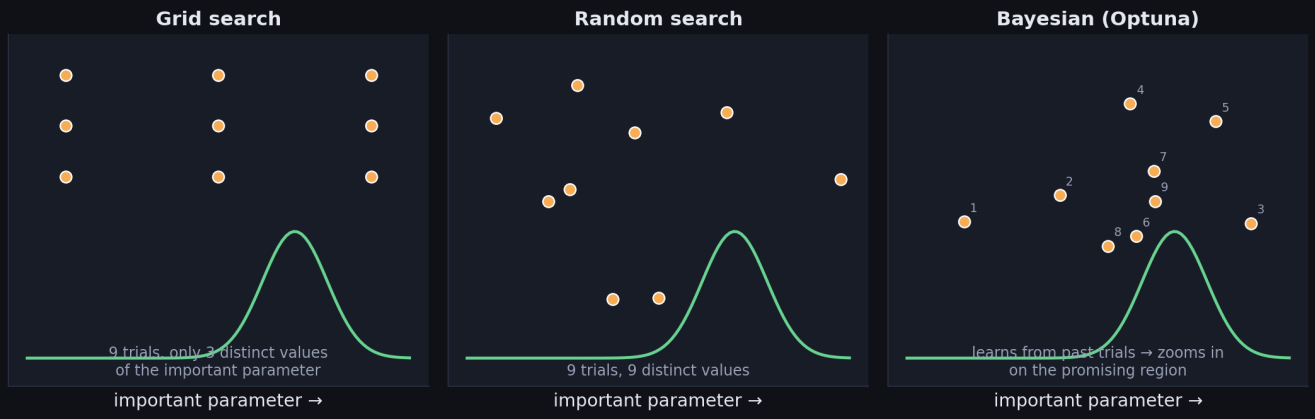


Figure 29 – Illustration: with 9 trials a grid explores only 3 values of the parameter that matters; random search explores 9; Bayesian search concentrates on the promising region.

optuna_search.py

```
import optuna, xgboost as xgb
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=2500, n_features=30, n_informative=6, flip_y=0.15, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
dtrain = xgb.DMatrix(X_train, label=y_train)

def objective(trial):
    params = {
        "objective": "binary:logistic", "eval_metric": "logloss", "tree_method": "hist",
        "eta": trial.suggest_float("eta", 0.01, 0.3, log=True),
        "max_depth": trial.suggest_int("max_depth", 3, 9),
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 10),
        "gamma": trial.suggest_float("gamma", 0, 1),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-3, 10, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 0.1, 10, log=True),
    }
    r = xgb.cv(params, dtrain, num_boost_round=2000, nfold=5, stratified=True,
               early_stopping_rounds=50, seed=42)
    trial.set_user_attr("n_estimators", len(r))          # remember the best number of trees
    return r["test-logloss-mean"].min()                  # Optuna minimises this

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=40)
print("Best CV log-loss:", study.best_value)
print("Best params     :", study.best_params, "| trees:", study.best_trial.user_attrs["n_estimators"])
```

5.4 Community tips — explained like you are five

Five popular tips from the XGBoost community, each translated into plain English with an analogy, then backed with the technical reason and code.

Tip 1 — Use DMatrix (speak the model's native language)




```
dtrain = xgb.DMatrix(X_train, label=y_train)
```

Figure 30 – Convert once, train fast.

ANALOGY Pandas = English, XGBoost = French

If you feed the model a Pandas DataFrame, it has to constantly “translate” the data behind the scenes, which slows everything down. Converting your data to a DMatrix first is like translating the document into French beforehand — the model can read it instantly and run much faster.

```
dtrain = xgb.DMatrix(X_train, label=y_train)   # convert ONCE
dval   = xgb.DMatrix(X_val,   label=y_val)
params = {"objective": "binary:logistic", "tree_method": "hist", "eta": 0.1}
booster = xgb.train(params, dtrain, num_boost_round=500,
                    evals=[(dval, "val")], early_stopping_rounds=50)
```

Technical reason: DMatrix is XGBoost's internal data structure (compressed, column-oriented, with pre-computed statistics). Re-using one DMatrix across many training runs (for example inside a tuning loop with `xgb.cv`) avoids repeating the conversion. The scikit-learn wrapper converts for you each time you call `.fit()`, which is convenient but repeats work.

Tip 2 — Use hist (sort into buckets instead of checking every penny)

ANALOGY Dividing a giant pile of coins

The default method (exact) looks at every single coin individually to find the mathematically perfect way to split the pile. This takes forever. The hist (histogram) method simply scoops the coins into rough buckets first (small, medium, large) and splits the buckets. It is up to ~50× faster and gets almost exactly the same result.

Tip 3 — Test a micro-batch (taste the soup before cooking a huge batch)

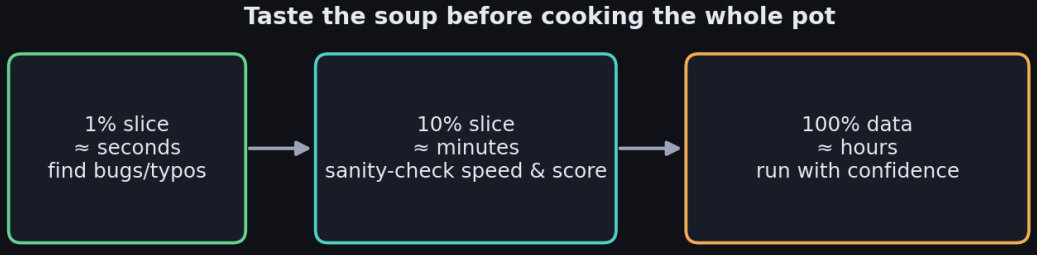


Figure 31 – Always scale up gradually.

Don't hit “run” on a massive dataset that takes 15 hours to process, only to have it crash at hour 14 because of a simple typo. Run your model on a tiny 1% slice of your data first. If it works on the small batch, you know it is safe to scale up to the big one.

```
X_small = X_train.sample(frac=0.01, random_state=0)      # 1% of rows (pandas)
y_small = y_train.loc[X_small.index]
```

```
xgb.XGBClassifier(tree_method="hist", n_estimators=20).fit(X_small, y_small)
# works?  -> try 10%, then 100%
```

Tip 4 — Avoid micro-learning rates (stop taking baby steps)

ANALOGY A marathon in baby steps

The learning rate is how big a step the model takes toward the final answer. If you set it microscopically low (like 0.001), you force the model to cross a marathon finish line by taking literal baby steps. It will take thousands of steps (trees) and hours of time. Start with normal strides (0.1) so the model finishes in minutes, not hours.

TIP Reconciling “lower is better” with “avoid tiny rates”

Both statements are true at different phases: **explore** with 0.05–0.1 (fast feedback while you tune everything else), then **finalise** with a smaller rate such as 0.01–0.03 plus early stopping if you can afford the time. Figure 14 shows why: very small rates need very many trees to pay off.

Tip 5 — Prune features (throw out the junk mail)

If you give the model 200 columns of data, but 50 of them are completely useless (like a column filled with zeros, or random ID numbers), you force the computer to read junk mail. Run a quick test, find out which columns the model ignores, and delete them. Less data to read means a faster model.

PART 6 · Complete Code (Level 4)

6.1 The Kaggle-style GridSearchCV template

A complete, runnable Python template using a parameter grid inspired by common Kaggle kernels, focusing on the “sweet spot” combinations that give high accuracy without taking days to compute. (We ran it: the output is shown below the code.)

kaggle_template.py

```
import xgboost as xgb
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

# 1. Create a dummy dataset (Replace this with your actual data)
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Initialize the XGBoost Classifier
# We use tree_method='hist' for speed, as recommended by the community
xgb_model = xgb.XGBClassifier(
    tree_method='hist',
    objective='binary:logistic',  # Use 'reg:squarederror' for regression
    random_state=42
)

# 3. Define the Kaggle-Optimized Parameter Grid
# This grid avoids exploring useless extremes to save time.
param_grid = {
    'learning_rate': [0.01, 0.05, 0.1],   # 0.05 to 0.1 is the Kaggle sweet spot for initial tuning
    'n_estimators': [100, 300, 500],      # Number of trees (lower learning rate needs more trees)
    'max_depth': [3, 5, 7],               # Tree depth: 3 to 7 usually captures all complex patterns
    'min_child_weight': [1, 3, 5],        # Controls overfitting: 1 is default, 3 and 5 are more conservative
    'subsample': [0.8, 1.0],              # 0.8 introduces randomness to prevent overfitting
    'colsample_bytree': [0.8, 1.0]        # 0.8 drops 20% of features per tree to force generalization
}

# 4. Set up GridSearchCV
# cv=3 or cv=5 is standard. n_jobs=-1 uses all your computer's CPU cores.
grid_search = GridSearchCV(
    estimator=xgb_model,
    param_grid=param_grid,
    scoring='accuracy',   # Use 'neg_mean_squared_error' for regression
    cv=3,
    n_jobs=-1,
    verbose=1             # Prints the progress so you don't stare at a blank screen
)

# 5. Run the Grid Search
print("Starting Grid Search... this may take a few minutes.")
grid_search.fit(X_train, y_train)

# 6. Extract the Best Model and Parameters
print("\n--- Grid Search Results ---")
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Best Cross-Validation Accuracy: {grid_search.best_score_:.4f}")

# 7. Make Predictions on Unseen Data
best_model = grid_search.best_estimator_
predictions = best_model.predict(X_test)
test_accuracy = accuracy_score(y_test, predictions)

print(f"Test Set Accuracy: {test_accuracy:.4f}")
```

What each step does

Step	What it does	Why
1 dummy dataset	make_classification builds 1000 rows × 20 features	replace with your own X, y
2 XGBClassifier	base model with tree_method='hist'	speed; objective='binary:logistic' for 2 classes ('reg:squarederror' for regression)
3 param_grid	6 hyperparameters with 2–3 values each	covers the most profitable ranges only
4 GridSearchCV	cv=3 folds, n_jobs=-1 (all CPU cores), verbose=1	each combo is trained 3 times and averaged
5 fit	runs the search	prints “Fitting 3 folds for each of 324 candidates, totalling 972 fits”
6 best_params_ / best_score_	best combination and its CV accuracy	what to use next
7 predict on X_test	final honesty check on unseen data	never tune on this set

MATH How many fits is that?

3 × 3 × 3 × 3 × 2 × 2 = 324 combinations, × 3 folds = **972 fits**. Our real run confirmed exactly “324 candidates, totalling 972 fits”. Add one more 3-value parameter and it triples.

actual output of the run in this tutorial (yours will differ with hardware/versions)

```
Fitting 3 folds for each of 324 candidates, totalling 972 fits

--- Grid Search Results ---
Best Parameters: {'colsample_bytree': 1.0, 'learning_rate': 0.05, 'max_depth': 3,
                  'min_child_weight': 5, 'n_estimators': 100, 'subsample': 1.0}
Best Cross-Validation Accuracy: 0.9088
Test Set Accuracy: 0.9050
```

Why this specific grid?

If you tested every possible parameter your computer would crash or run for weeks. Experienced Kaggle competitors use grids like this because they target the most profitable ranges:

- **The Depth Rule:** they rarely test max_depth above 7. If an XGBoost model needs a depth of 10 to learn, it is almost certainly memorising the training data (overfitting).
- **The 0.8 Rule:** setting subsample and colsample_bytree to 0.8 is a popular Kaggle trick. It forces the model to ignore 20% of the rows and 20% of the columns every time it builds a tree, forcing it to find hidden patterns instead of relying on the easiest, most obvious data points.
- **The Learning Rate Balance:** they pair 0.1 learning rates with ~100 trees, and 0.01 learning rates with 500+ trees (see the diagonal in Figure 14).

WATCH OUT Rules of thumb, not laws

“Never above depth 7” and “0.8 is magic” are good starting points that come from experience on typical tabular data. Your dataset can disagree — always let validation scores decide.

6.2 The pro workflow: staged tuning + early stopping + DMatrix

This script combines everything from Part 5: one DMatrix, 5-fold cross-validated early stopping, four stages. It evaluates about 36 small CV runs instead of thousands of fits. (Tested: it ran in under a minute on the toy data and reached test log-loss 0.389 / accuracy 0.862.)

staged_tuning.py

```
import numpy as np, xgboost as xgb
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import log_loss, accuracy_score

X, y = make_classification(n_samples=2500, n_features=30, n_informative=6, flip_y=0.15, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
dtrain = xgb.DMatrix(X_train, label=y_train)            # convert ONCE (speaks XGBoost's native language)

params = {"objective": "binary:logistic", "eval_metric": "logloss", "tree_method": "hist",
          "eta": 0.1, "max_depth": 6, "min_child_weight": 1, "subsample": 0.8,
          "colsample_bytree": 0.8, "gamma": 0, "seed": 42}

def cv_score(p):
    """5-fold CV with early stopping -> (best number of trees, best CV log-loss)."""
    r = xgb.cv(p, dtrain, num_boost_round=2000, nfold=5, stratified=True,
               early_stopping_rounds=50, seed=42)
    return len(r), r["test-logloss-mean"].min()

# STAGE 1: fixed learning rate -> let early stopping find n_estimators
rounds, score = cv_score(params)
print(f"Stage 1: {rounds} trees, CV logloss {score:.4f}")

# STAGE 2: max_depth + min_child_weight together (16 combinations)
best = (score, params["max_depth"], params["min_child_weight"])
for d in [3, 5, 7, 9]:
    for mcw in [1, 3, 5, 7]:
        _, s = cv_score({**params, "max_depth": d, "min_child_weight": mcw})
        if s < best[0]: best = (s, d, mcw)
params.update(max_depth=best[1], min_child_weight=best[2])
print("Stage 2 ->", best)

# STAGE 3: sampling parameters (9 combinations)
best = (1e9, 0.8, 0.8)
for ss in [0.7, 0.8, 0.9]:
    for cs in [0.7, 0.8, 0.9]:
        _, s = cv_score({**params, "subsample": ss, "colsample_bytree": cs})
        if s < best[0]: best = (s, ss, cs)
params.update(subsample=best[1], colsample_bytree=best[2])
print("Stage 3 ->", best)

# STAGE 4: lower the learning rate, re-run early stopping, train the final model
params["eta"] = 0.02
rounds, score = cv_score(params)
final = xgb.train(params, dtrain, num_boost_round=rounds)
p = final.predict(xgb.DMatrix(X_test))
print(f"Stage 4: {rounds} trees | test logloss {log_loss(y_test, p):.4f} | test acc {accuracy_score(y_test, p > 0.5):.3f}")
```

6.3 Looking inside the final model

```
import matplotlib.pyplot as plt
xgb.plot_importance(final, importance_type="gain", max_num_features=15)
plt.show()

# raw trees as text: see the actual questions and leaf weights
print(final.get_dump()[0])        # the first tree

# how many trees did early stopping keep?
print(final.num_boosted_rounds())
```

PART 7 · Cheat Sheets & Practice

7.1 Troubleshooting table

Symptom	Diagnosis	What to change
Train score great, validation poor	overfitting	lower max_depth (3–6); raise min_child_weight; add gamma 0.1–0.3; subsample/colsample 0.7–0.9; raise reg_lambda; lower learning_rate with early stopping
Both scores poor	underfitting	raise max_depth; more trees; lower min_child_weight / gamma / reg_*; check features
Validation worsens as trees increase	too many trees	early_stopping_rounds=50
Training too slow	too much compute	tree_method='hist' (+ device='cuda'), learning_rate ≈ 0.1, fewer features, DMatrix, smaller grid
Rare class never predicted	imbalance	scale_pos_weight = neg/pos; PR-AUC; tune threshold
Many NaNs	(not a problem)	do NOT impute; pass NaN directly
Huge number of useless columns	noise + slowness	drop bottom importance features; raise reg_alpha; lower colsample_bytree

7.2 Recommended starting recipe

a sensible first model

```
xgb.XGBClassifier(
    tree_method="hist",
    n_estimators=2000, early_stopping_rounds=50,   # let early stopping choose
    learning_rate=0.05,
    max_depth=5,
    min_child_weight=3,
    subsample=0.8, colsample_bytree=0.8,
    gamma=0, reg_alpha=0, reg_lambda=1,
    eval_metric="logloss", random_state=42,
)
```

7.3 The ten golden rules

1. Split data into train / validation / test. Never tune on test.
2. Start with defaults and measure a baseline.
3. Use tree_method='hist'.
4. Fix learning_rate ≈ 0.05–0.1 and let early stopping choose n_estimators.
5. Tune max_depth and min_child_weight together.
6. Then tune subsample and colsample_bytree.
7. Add gamma / reg_alpha / reg_lambda only if still overfitting.
8. Tune in stages; use RandomizedSearchCV or Optuna instead of giant grids.
9. Lower the learning rate at the very end for the final model.
10. Debug on a 1% slice before running on everything.

7.4 Glossary

Term	Meaning
Boosting	building models sequentially, each correcting the previous ones
Residual	true value − current prediction
Leaf weight	number stored in a leaf; the tree's output for rows reaching it
Gradient (g)	first derivative of the loss; says which direction to move
Hessian (h)	second derivative of the loss; curvature, controls step confidence
Regularisation	penalty that discourages complex models
Overfitting	good on training data, bad on unseen data
Early stopping	stop adding trees when validation score stops improving
Cross-validation	splitting training data into folds to estimate performance reliably
Hyperparameter	setting chosen by you before training

7.5 Practice exercises (do these to really learn)

1. Re-create Figure 15: plot train vs test loss for max_depth 1…12 on your own dataset. At what depth does test loss turn up?
2. Run the same training with learning_rate 0.3, 0.1, 0.01 using early stopping. Compare the number of trees chosen.
3. Add random-ID and all-zero columns to your data. Check whether feature importance gives them low scores.
4. Use scale_pos_weight on an imbalanced dataset and plot the precision–recall curve before and after.
5. Change the TinyXGB code to use gamma=5 and see how many leaves your trees lose.
6. Run the Optuna script for 40 trials and compare its best score with the Kaggle grid at equal compute time.

> **TIP** **You made it!**
>
> You started with “what is a decision tree” and finished with second-order Taylor expansions, hand-built boosting, GPU/histogram speed-ups and Bayesian tuning. The best next step: take one real dataset and apply the staged workflow from Part 5 — that is how this knowledge becomes skill.